

CELEBAL TECHNOLOGIES

Data Engineering Internship

Week 5 Assignment

Apache Spark — DataFrames, Transformations & Data Cleaning

Submitted By: Harshit Sharma

Role: Data Engineering Intern

CEI Program OF 2026

Q1. Key Limitations of Traditional MapReduce That Make Spark Preferred:-

MapReduce was revolutionary when it was introduced, but it has some fundamental design choices that make it very slow for modern workloads — especially when we need to run iterative algorithms or interactive queries. Here's what makes Spark a better fit:

Limitation in MapReduce	How Spark Addresses It
Every intermediate result is written to HDFS (disk I/O is the bottleneck)	Spark keeps intermediate data in memory (RAM) using RDDs/DataFrames, drastically reducing I/O
A job is locked into two phases: Map → Reduce. Complex pipelines require multiple chained jobs	Spark supports a rich DAG of transformations (filter, join, agg, etc.) all within a single job
No native support for streaming. You'd need separate tools like Storm	Spark Streaming / Structured Streaming handles real-time data in the same API
Each MapReduce iteration re-reads data from disk — terrible for ML training loops	Spark can cache datasets in memory, making repeated passes 10–100x faster
High latency startup time per job; not suitable for exploratory queries	Spark's interactive shell (spark-shell, PySpark REPL) gives near-real-time feedback
Writing MapReduce code requires verbose Java; even simple tasks are complex	Spark's DataFrame/SQL API is concise and expressive, similar to Pandas or SQL

In practice, when training a machine learning model that needs 100 iterations over the same dataset, MapReduce would read from disk 100 times. Spark reads the data once, caches it, and iterates entirely in memory. That is the core reason companies like Uber, Netflix, and Airbnb migrated from Hadoop MapReduce to Spark.

Q2. How Spark Uses In-Memory Computing to Speed Up Iterative ML Algorithms:-

To really understand this, let's trace what happens in both systems when we train a logistic regression model that needs 50 epochs:

Disk-Based System (MapReduce):

- Read training data from HDFS → Map phase → Write intermediate output → Reduce → Write final output to HDFS
- For iteration 2: Read again from HDFS → Map → Write → Reduce → Write...
- 50 iterations = 50 full disk reads + 50 writes. Network + disk I/O dominates runtime

Spark In-Memory Approach:

- Read training data from HDFS into an RDD/DataFrame
- Call `.cache()` or `.persist()` to pin this data in executor memory
- Iteration 1: compute gradients from in-memory data
- Iteration 2–50: data is already in RAM — no disk touch at all
- Result: the only disk I/O is the initial load and the final write

Spark models computation as a Directed Acyclic Graph (DAG). When you call `.cache()`, Spark flags that RDD/DataFrame partition to stay in executor JVM heap. Subsequent actions that need the same data hit memory directly. This is why Spark is often quoted as being 100x faster than MapReduce for iterative workloads.

```
# Example: caching a training dataset for ML iteration
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('MLExample').getOrCreate()

# Load once from storage
training_df = spark.read.parquet('hdfs://data/training/')

# Cache in memory - subsequent iterations skip disk
training_df.cache()

# Iterative training loop (e.g., gradient descent)
for epoch in range(50):
    loss = compute_loss(training_df)    # reads from memory, not disk
    update_weights(loss)

# Free memory when done
training_df.unpersist()
```

Spark also provides different storage levels via `.persist()`: `MEMORY_ONLY`, `MEMORY_AND_DISK`, `DISK_ONLY`, etc. This lets you tune based on how much RAM is available — if an executor runs out of memory, Spark gracefully spills to disk rather than crashing.

Q3. Removing Duplicate Rows Based on Specific Columns

When deduplicating, you rarely want to drop rows that differ on even one column — usually you want to say 'if two rows have the same `user_id` and `transaction_date`, treat them as duplicates'. Spark's `.dropDuplicates()` method accepts a subset of columns for exactly this purpose:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('DeduplicationExample').getOrCreate()

# Sample DataFrame with intentional duplicates
data = [
    (1, '2024-01-15', 'Amazon', 250.00),
    (1, '2024-01-15', 'Amazon', 250.00),    # exact duplicate
    (1, '2024-01-15', 'Flipkart', 100.00), # same user+date, diff merchant
    (2, '2024-01-16', 'Amazon', 300.00),
]
columns = ['user_id', 'transaction_date', 'merchant', 'amount']
df = spark.createDataFrame(data, columns)

# Drop duplicates based on user_id and transaction_date only
# Keeps the FIRST occurrence of each (user_id, transaction_date) pair
df_deduped = df.dropDuplicates(['user_id', 'transaction_date'])

df_deduped.show()
# Output:
# +-----+-----+-----+-----+
# |user_id|transaction_date|merchant|amount|
```

```
# +-----+-----+-----+-----+
# |      1|      2024-01-15|  Amazon|250.00|  <-- first occurrence kept
# |      2|      2024-01-16|  Amazon|300.00|
# +-----+-----+-----+-----+

# Count before and after to verify
print(f'Before: {df.count()} rows, After: {df_deduped.count()} rows')
```

Note: `.dropDuplicates()` without any arguments removes only rows that are completely identical across every column. Passing a subset list makes the deduplication logic explicit and intentional — which is almost always what you want in a data pipeline.

Q4. Filter by Region and Group By Product Category to Find Average Sale Amount :-

This is a two-step transformation: first narrow down the data to only West region rows, then aggregate by `product_category`. Both the DataFrame API and Spark SQL work equally well here — let me show both:

Approach 1 — DataFrame API :-

```
from pyspark.sql import functions as F

result = (
    df_sales
    .filter(F.col('region') == 'West')           # Step 1: filter
    .groupBy('product_category')                 # Step 2: group
    .agg(F.avg('sale_amount').alias('avg_sale')) # Step 3: aggregate
    .orderBy(F.desc('avg_sale'))                 # Optional: sort by highest avg
)

result.show()
```

Approach 2 — Spark SQL (more readable for complex queries) :-

```
df_sales.createOrReplaceTempView('sales')

result = spark.sql('''
    SELECT
        product_category,
        AVG(sale_amount) AS avg_sale
    FROM sales
    WHERE region = 'West'
    GROUP BY product_category
    ORDER BY avg_sale DESC
''')

result.show()
```

Both produce identical results. The DataFrame API is preferred in production pipelines (easier to compose programmatically), while Spark SQL is great for ad-hoc exploration or when business users are comfortable with SQL syntax.

Q5. Difference Between .na.drop() and .na.fill() with Code Example

These two are complementary null-handling strategies with very different outcomes:

Method	What It Does	Use When
.na.drop()	Removes entire rows that have null values in any (or specified) columns	You cannot infer a meaningful replacement and the row is unusable without that value
.na.fill()	Replaces null values with a specified default value	You have a sensible default (e.g., 'Unknown', 0, -1) and want to retain the row

Code Example — Filling nulls in the 'status' column with 'Unknown':

```
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName('NullHandling').getOrCreate()

# Sample data with nulls
data = [
    (1, 'Alice', 'Active'),
    (2, 'Bob', None),      # null status
    (3, None, 'Inactive'),
    (4, 'Diana', None),   # null status
]
df = spark.createDataFrame(data, ['id', 'name', 'status'])

# .na.fill() - replace nulls in 'status' column only
df_filled = df.na.fill({'status': 'Unknown'})
df_filled.show()
# +---+-----+-----+
# | id| name|  status|
# +---+-----+-----+
# | 1|Alice|  Active|
# | 2|  Bob| Unknown|    <-- filled
# | 3| null|Inactive|    <-- name still null (not touched)
# | 4|Diana| Unknown|    <-- filled
# +---+-----+-----+

# .na.drop() - drop any row where 'name' is null
df_dropped = df.na.drop(subset=['name'])
df_dropped.show()
# +---+-----+-----+
# | id| name|  status|
# +---+-----+-----+
# | 1|Alice|  Active|
# | 2|  Bob|    null|
# | 4|Diana|    null|
# +---+-----+-----+ (row 3 with null name is gone)
```

Q6. Count Records Per City, Only Where Count > 100

This requires `groupBy` + `count`, followed by a filter on the aggregated result — which in SQL terminology is a `HAVING` clause. In Spark's `DataFrame` API, you apply `.filter()` after `.agg()`:

DataFrame API:

```
from pyspark.sql import functions as F

city_counts = (
    df
    .groupBy('city')
    .agg(F.count('*').alias('record_count'))
    .filter(F.col('record_count') > 100)          # equivalent to SQL HAVING
    .orderBy(F.desc('record_count'))
)

city_counts.show()
```

Spark SQL equivalent (using HAVING):

```
df.createOrReplaceTempView('records')

spark.sql('''
    SELECT city, COUNT(*) AS record_count
    FROM records
    GROUP BY city
    HAVING COUNT(*) > 100
    ORDER BY record_count DESC
''').show()
```

A common mistake is trying to use `.filter()` before `.groupBy()` on the aggregated column — that won't work because the aggregation hasn't happened yet. Always filter on aggregated columns after the `.agg()` call.

Q7. How Immutability of Spark DataFrames Affects Data Cleaning

Spark `DataFrames` are immutable — once created, their content and schema cannot be changed in place. Any transformation you apply (`drop`, `rename`, `filter`, `cast`) does not modify the original; instead, it returns a brand new `DataFrame`. This design choice has important implications for how you write data cleaning code:

What immutability means practically:

```
# WRONG approach - many beginners do this and wonder why nothing changed
df.drop('unwanted_column')          # This does nothing to df
df.withColumnRenamed('old', 'new')  # df is unchanged

# CORRECT approach - always capture the result
df_clean = df.drop('unwanted_column')
df_clean = df_clean.withColumnRenamed('old_name', 'clean_name')
```

Chaining transformations (most common pattern in production):

```
df_clean = (
    df
    .drop('redundant_col', 'temp_col')          # Step 1: remove unwanted cols
```

```

.withColumnRenamed('cust_nm', 'customer_name') # Step 2: rename
.withColumnRenamed('amt', 'amount')
.filter(F.col('amount') > 0) # Step 3: filter bad data
.na.fill({'status': 'Unknown'}) # Step 4: handle nulls
)

```

The upside of immutability is reproducibility and safety: you can always reference the original df at any point in your pipeline without worrying that a downstream step corrupted it. It also enables Spark's lazy evaluation model — Spark builds up a DAG of transformation steps and optimizes them all before running anything.

In data cleaning pipelines, it is common practice to keep the original raw DataFrame intact and build a separate df_clean through a chain of transformations. This lets you compare raw vs. cleaned data and re-run any step independently if needed.

Q8. Filter Rows Where Age is Between 18–30 and Subscription is 'Premium'

This combines a range condition on a numeric column with an equality check on a string column. Spark supports multiple syntax styles — all are valid, pick the one that feels clearest:

Method 1 — Using .between() (most readable):

```

from pyspark.sql import functions as F

df_filtered = df.filter(
    F.col('age').between(18, 30) &
    (F.col('subscription') == 'Premium')
)

df_filtered.show()

```

Method 2 — Explicit comparison operators:

```

df_filtered = df.filter(
    (F.col('age') >= 18) &
    (F.col('age') <= 30) &
    (F.col('subscription') == 'Premium')
)

```

Method 3 — Spark SQL string expression:

```

df_filtered = df.filter("age BETWEEN 18 AND 30 AND subscription = 'Premium'")

```

Important: Always wrap each condition in parentheses when combining with & or | operators in Python. Without parentheses, Python operator precedence can silently produce wrong results because & binds tighter than comparison operators.

Q9. Why Handle Null Values Before Mathematical Aggregations Like sum() or avg()

Spark follows SQL semantics for null handling: any arithmetic operation involving null propagates null. This has a critical consequence for aggregations:

How Spark's sum() and avg() handle nulls:

```
# Sample: total_price has a null
data = [(1, 500.0), (2, None), (3, 300.0), (4, None), (5, 200.0)]
df = spark.createDataFrame(data, ['order_id', 'total_price'])

# sum() ignores nulls - gives 1000.0 (not 0 or error)
df.agg(F.sum('total_price')).show()    # Result: 1000.0

# avg() also ignores nulls - averages only non-null rows
# avg = (500 + 300 + 200) / 3 = 333.33 - NOT (1000 / 5 = 200)
df.agg(F.avg('total_price')).show()    # Result: 333.33
```

The problem above is that avg() divides by 3 (non-null count) instead of 5 (total rows). If your business requirement says 'null price means 0 (free item)', the average should be 200 — but Spark gives 333. This silent statistical distortion is dangerous because it looks correct but isn't.

Correct approach — fill nulls first based on business logic:

```
# If null price means the item was free (price = 0)
df_clean = df.na.fill({'total_price': 0})
df_clean.agg(F.avg('total_price')).show()    # Correct: 200.0

# If null price means data is missing and row should be excluded
df_clean = df.na.drop(subset=['total_price'])
df_clean.agg(F.avg('total_price')).show()    # Explicit exclusion: 333.33
```

The key point is intentionality. Handling nulls before aggregations forces you to make a deliberate decision about what nulls mean in your domain — free items, missing data, not applicable — rather than letting the engine silently decide for you.

Q10. Cast raw_timestamp to TimestampType and Rename to event_time

This is a two-step column transformation: first cast the data type, then rename. Both can be chained in a single .withColumn() + .withColumnRenamed() or handled in one shot using .select():

Approach 1 — withColumn + withColumnRenamed (explicit, readable):

```
from pyspark.sql import functions as F
from pyspark.sql.types import TimestampType

df_updated = (
    df
    .withColumn('raw_timestamp', F.col('raw_timestamp').cast(TimestampType()))
    .withColumnRenamed('raw_timestamp', 'event_time')
)

df_updated.printSchema()
# root
# |-- event_time: timestamp (nullable = true)
```


Approach 2 — Single withColumn with alias (cleaner one-liner):

```
df_updated = df.withColumn(  
    'event_time',  
    F.col('raw_timestamp').cast('timestamp')  
) .drop('raw_timestamp') # Remove the original column
```

Approach 3 — Using select with alias (best for when you're restructuring multiple columns):

```
df_updated = df.select(  
    F.col('raw_timestamp').cast('timestamp').alias('event_time'),  
    '*' # Keep all other columns (note: raw_timestamp must be excluded separately if  
needed)  
)
```

Approach 1 is generally preferred in production because the intent is crystal clear at each step. The cast string 'timestamp' is equivalent to TimestampType() — both work. If your source data is a string like '2024-01-15 10:30:00', Spark will parse it automatically when casting to timestamp.

Q11. The Shuffle Process During Grouping — Why It Is a Wide Transformation

This is one of the most important performance concepts in Spark. To understand shuffle, you first need to understand how Spark partitions data across a cluster.

Background — Narrow vs. Wide Transformations:

Type	Definition	Examples	Shuffle?
Narrow	Each output partition depends on only one input partition	map(), filter(), union()	No
Wide	Each output partition may depend on multiple input partitions	groupBy(), join(), distinct()	Yes — triggers Shuffle

What happens during a groupBy shuffle:

- Spark splits your data into N partitions distributed across executor nodes. Partition 1 might be on Node A, Partition 2 on Node B, etc.
- When you call .groupBy('city'), Spark needs to collect all rows for 'Mumbai' onto the same partition — but they could be on any node right now.
- Spark computes a hash of the group key (e.g., hash('Mumbai') % numPartitions) to determine the target partition.
- Every executor sends (writes to disk) its local rows to the correct target partitions across the network — this is the shuffle write phase.
- Target executors then fetch (read from disk) their assigned rows from all senders — this is the shuffle read phase.
- Once all rows for 'Mumbai' are co-located, the aggregation (sum, count, avg) happens locally.

Why shuffle is expensive:

- It involves network I/O (data crosses machine boundaries)
- Shuffle data is written to disk as intermediate files (shuffle files), causing disk I/O

- All stages after the shuffle must wait for all shuffle writes to complete — this is a synchronization barrier
- Skewed data (one key with far more rows than others) causes a single executor to become a bottleneck

```
# You can control the number of shuffle partitions
# Default is 200 — often too many for small datasets, too few for large ones
spark.conf.set('spark.sql.shuffle.partitions', '50') # tune based on data size

# Check how many partitions after a groupBy
result = df.groupBy('city').count()
print(result.rdd.getNumPartitions()) # Will equal shuffle.partitions setting
```

Q12. Remove Rows Where email is Null OR username is an Empty String

This needs a combined filter that handles two different kinds of 'bad data' — actual nulls and empty strings (which are not the same thing in Spark). The row should be dropped if either condition is true, so we use OR logic:

```
from pyspark.sql import functions as F

# Remove rows where email IS NULL or username is empty string
df_clean = df.filter(
    F.col('email').isNull() &
    (F.trim(F.col('username')) != '')
)

# Alternative using ~ (NOT) with the bad condition:
df_clean = df.filter(
    ~(F.col('email').isNull() | (F.trim(F.col('username')) == ''))
)

# Verify the removal
print(f'Original rows: {df.count()}')
print(f'Cleaned rows: {df_clean.count()}')
print(f'Removed: {df.count() - df_clean.count()}')
```

Note the use of `F.trim()` around username — this also catches usernames that look non-empty but contain only whitespace (e.g., ' '). Without trim, a username of ' ' would pass the empty string check and sneak through. This is a common data quality gotcha in real pipelines.

If you also want to treat 'None' as a string (some sources write the word None):

```
df_clean = df.filter(
    F.col('email').isNull() &
    F.col('email').isin(['None', '']) == False & # guard against string 'None'
    (F.trim(F.col('username')) != '') &
    (F.col('username') != 'None')
)
```

Q13. Using .agg() to Calculate min, max, and mean of the price Column

The `.agg()` function is Spark's way of computing multiple aggregate statistics in one pass over the data — much more efficient than running separate `groupBy + count`, `groupBy + avg`, etc. queries:

Basic usage — multiple stats in one call:

```
from pyspark.sql import functions as F

# Method 1: Using F.min, F.max, F.mean with aliases
stats = df.agg(
    F.min('price').alias('min_price'),
    F.max('price').alias('max_price'),
    F.mean('price').alias('mean_price'),
    F.count('price').alias('total_records'),
    F.stddev('price').alias('std_price') # bonus: standard deviation
)

stats.show()
# +-----+-----+-----+-----+-----+
# |min_price|max_price|mean_price|total_records|std_price|
# +-----+-----+-----+-----+-----+
# |    10.99|    999.99|    149.75|         5000|    87.32|
# +-----+-----+-----+-----+-----+
```

Combined with groupBy — stats per category:

```
# Stats broken down by product category
category_stats = (
    df
    .groupBy('product_category')
    .agg(
        F.min('price').alias('min_price'),
        F.max('price').alias('max_price'),
        F.mean('price').alias('avg_price'),
        F.count('*').alias('item_count')
    )
    .orderBy('product_category')
)

category_stats.show()
```

The key advantage of `.agg()` is that all statistics are computed in a single distributed pass. Calling `df.select(F.min('price'))`, `df.select(F.max('price'))`, `df.select(F.mean('price'))` separately would trigger three separate Spark jobs — three times the overhead.

Q14. Risk of Using `inferSchema=true` with Messy or Inconsistent Date Formats

Schema inference seems convenient — Spark reads a sample of the file and guesses the data types for you. But when date/timestamp columns contain inconsistent formats, `inferSchema=true` can silently produce wrong results or cause data loss. Here's why:

How `inferSchema` works with dates:

- Spark reads a sample of rows (typically the first few thousand)
- If those sample rows contain '2024-01-15' (ISO format), Spark infers `TimestampType` or `DateType`
- If later rows have '15/01/2024' (DD/MM/YYYY) or '01-15-2024' (US format), Spark cannot parse them with the inferred type
- Result: those rows get silently cast to null — no error thrown, data just disappears

Concrete example of the problem:

```
# CSV file with inconsistent date formats
# Row 1: 2024-01-15 <- ISO format (what Spark samples)
# Row 2: 15/01/2024 <- DD/MM/YYYY
# Row 3: Jan 15 2024 <- Long format

# With inferSchema=True:
df = spark.read.option('inferSchema', 'true').csv('data.csv', header=True)
# Spark infers 'event_date' as DateType based on sample
# Rows with '15/01/2024' become null — no warning!

df.filter(F.col('event_date').isNull()).count() # Silently non-zero!
```

Correct approach — define the schema explicitly:

```
from pyspark.sql.types import StructType, StructField, StringType
from pyspark.sql import functions as F
```

```
schema = StructType([
    StructField('order_id', StringType(), True),
    StructField('event_date', StringType(), True),
    StructField('amount', StringType(), True),
])

df = spark.read.schema(schema).csv('data.csv', header=True)

df_clean = df.withColumn(
    'event_date_parsed',
    F.coalesce(
        F.to_date(F.col('event_date'), 'yyyy-MM-dd'),
        F.to_date(F.col('event_date'), 'dd/MM/yyyy'),
        F.to_date(F.col('event_date'), 'MM-dd-yyyy')
    )
)

df_clean.show()
```

This explicit approach gives you control. You can log rows where none of the formats matched (all coalesce results are null) and investigate them — rather than silently dropping data. Always prefer explicit schema definition in production data pipelines.

Q15. Final Processing Pipeline: Deduplicate → Fill Null Prices → Group by store_id

This question asks for an end-to-end mini-pipeline that demonstrates three core data engineering operations in sequence. Let's build it step by step and then show the complete chained version:

Step-by-step breakdown:

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName('RevenuePipeline').getOrCreate()

# Sample raw data — has duplicates and null prices
```

```

raw_data = [
    ('S001', 'P101', 150.0),
    ('S001', 'P101', 150.0), # duplicate
    ('S002', 'P201', None), # null price
    ('S002', 'P202', 200.0),
    ('S003', 'P301', 350.0),
    ('S001', 'P102', None), # null price
]
df_raw = spark.createDataFrame(raw_data, ['store_id', 'product_id', 'price'])

print('=== Raw Data ===')
df_raw.show()

```

```

# STEP 1: Remove duplicate rows (exact match across all columns)
df_step1 = df_raw.dropDuplicates()
print(f'After dedup: {df_step1.count()} rows (was {df_raw.count()})')

```

```

# STEP 2: Fill null prices with 0
df_step2 = df_step1.na.fill({'price': 0})

```

```

# STEP 3: Group by store_id and calculate total revenue
df_result = (
    df_step2
    .groupBy('store_id')
    .agg(
        F.sum('price').alias('total_revenue'),
        F.count('*').alias('transaction_count'),
        F.avg('price').alias('avg_price_per_txn')
    )
    .orderBy('store_id')
)

```

```

print('=== Final Revenue by Store ===')
df_result.show()

```

```

# Expected output:
# +-----+-----+-----+-----+
# |store_id|total_revenue|transaction_count| avg_price_per_txn|
# +-----+-----+-----+-----+
# |    S001|         150.0|                2|          75.0| (1 deduped + null=0)
# |    S002|         200.0|                2|         100.0| (null=0 + 200)
# |    S003|         350.0|                1|         350.0|
# +-----+-----+-----+-----+

```

Production-grade version — fully chained with logging:

```

def run_revenue_pipeline(df_raw):
    """
    End-to-end revenue pipeline:
    1. Remove duplicates
    2. Fill null prices with 0
    3. Aggregate total revenue by store
    """
    raw_count = df_raw.count()

    df_result = (
        df_raw
        # Step 1: Deduplicate
        .dropDuplicates()
    )

```

```

# Step 2: Fill nulls
.na.fill({'price': 0})
# Step 3: Group and aggregate
.groupby('store_id')
.agg(
    F.sum('price').alias('total_revenue'),
    F.count('*').alias('transaction_count')
)
.orderBy(F.desc('total_revenue')) # highest revenue store first
)

final_count = df_result.count()
print(f'Pipeline complete: {raw_count} raw rows -> {final_count} store summaries')
return df_result

# Run it
output = run_revenue_pipeline(df_raw)
output.show()

# Optionally save to Parquet for downstream consumption
output.write.mode('overwrite').parquet('output/store_revenue/')

```

Wrapping the pipeline in a function is best practice — it makes the code testable (you can pass in mock DataFrames), reusable, and easy to schedule in tools like Apache Airflow. The `.dropDuplicates()` → `.na.fill()` → `.groupBy().agg()` pattern covers the vast majority of real-world data cleaning and aggregation use cases.

— End of Week 5 Assignment Submission —

Celebal Technologies| CEI Program | Data Engineering Internship | 2026